-TABLA DE CASOS

Casos normales

Casos	Entrada	Resultado Esperado	Regla	Justificación
1	7 min, normal, boleto no perdido	$0.00	R5	De 1 a 15 minutos es gratis
2	71 min, normal, boleto no perdido	$35.00	R7	$20 de la primer hora más $15 del inicio de la segunda
3	23 min, normal, boleto no perdido	$20.00	R6	De 16 a 60 minutos son $20

Casos frontera

Casos	Entrada	Resultado Esperado	Regla	Justificación
4	0 min, normal, boleto no perdido	$0.00	R4	0 minutos es gratis
5	15 min, normal, boleto no perdido	$0.00	R5	De 1 a 15 minutos es gratis
6	16 min, normal, boleto no perdido	$20.00	R6	De 16 a 60 minutos son $20
7	60 min, normal, boleto no perdido	$20.00	R6	De 16 a 60 minutos son $20
8	61 min, normal, boleto no perdido	$35.00	R7	$20 de la primer hora más $15 del inicio de la segunda
9	121 min, normal, boleto no perdido	$50.00	R7	$20 de la primer hora más $15 de la segunda más 15 del inicio de la tercera

Entradas inválidas

Casos	Entrada	Resultado Esperado	Reglas	Justificación
10	-3 min, normal, boleto no perdido	ERROR	R1 y R10	Minutos negativos
11	47 min, Batman, boleto no perdido	ERROR	R2 y R11	Tipo de cliente no válido

Interacción entre reglas

Casos	Entrada	Resultado Esperado	Regla	Justificación
12	43 min, frecuente, boleto no perdido	$18.00	R6 y R8	De 16 a 60 minutos es $20 menos el 10% por cliente frecuente
13	79 min, frecuente, boleto perdido	$300.00	R3 y R8	Boleto perdido sin importar el tipo de cliente

Números decimales

Casos	Entrada	Resultado Esperado	Reglas	Justificación
14	97 min, frecuente, boleto no perdido	$31.50	R7 y R8	$20 de la primer hora más $15 del inicio de la segunda menos 10% por cliente frecuente

Caso adicional

Casos	Entrada	Resultado Esperado	Reglas	Justificación
15	0 min, normal, boleto perdido	$300.00	R3 y R8	Boleto perdido, entonces aunque sean $0 por estar 0 minutos, los $300 siguen teniendo prioridad

-REGLAS CONSIDERADAS

R1. Los minutos deben ser un número entero mayor o igual que 0.
R2. Los tipos de cliente válidos son:

"normal"
"frecuente"

R3. Si el boleto está perdido, el costo final es de $300.00, sin importar los minutos ni el tipo de cliente.
R4. Si los minutos son 0, el costo es $0.00.
R5. De 1 a 15 minutos, el costo es $0.00.
R6. De 16 a 60 minutos, el costo es $20.00.
R7. Después de 60 minutos se cobran $20.00 por la primera hora y $15.00 por cada hora adicional
iniciada.
R8. Un cliente "frecuente" recibe 10 % de descuento sobre el costo calculado, excepto cuando perdió
el boleto.
R9. El costo nunca puede ser negativo.
R10. Si los minutos son negativos, el sistema debe lanzar un error.
R11. Si el tipo de cliente no es válido, el sistema debe lanzar un error.

-EVIDENCIAS DE EJECUCIÓN:

```
pytest -v
============================================================ test session
starts =============================================================
platform linux -- Python 3.13.13, pytest-8.4.2, pluggy-1.6.0 --
/home/suemy/Documentos/Modelado/Practica4-Pruebas-Unitarias/pytho
n/venv/bin/python3
cachedir: .pytest_cache
rootdir:
/home/suemy/Documentos/Modelado/Practica4-Pruebas-Unitarias/python
collected 21 items

test_estacionamiento.py::test_7_minutos_gratis PASSED
[  4%]
test_estacionamiento.py::test_71_minutos_35 FAILED
[  9%]
test_estacionamiento.py::test_23_minutos_20 PASSED
[ 14%]
test_estacionamiento.py::test_0_minutos_gratis PASSED
[ 19%]
test_estacionamiento.py::test_15_minutos_gratis FAILED
[ 23%]
test_estacionamiento.py::test_16_minutos_costo_20 PASSED
[ 28%]
test_estacionamiento.py::test_60_minutos_costo_20 PASSED
[ 33%]
```

```
test_estacionamiento.py::test_61_minutos_costo_35 FAILED
[ 38%]
test_estacionamiento.py::test_121_minutos_costo_50 FAILED
[ 42%]
test_estacionamiento.py::test_minutos_negativos PASSED
[ 47%]
test_estacionamiento.py::test_tipo_usuario_invalido PASSED
[ 52%]
test_estacionamiento.py::test_cliente_frecuente_costo_18 PASSED
[ 57%]
test_estacionamiento.py::test_cliente_frecuente_boleto_perdido
FAILED                                                    [ 61%]
test_estacionamiento.py::test_costo_con_decimales FAILED
[ 66%]
test_estacionamiento.py::test_0_minutos_boleto_perdido PASSED
[ 71%]
test_estacionamiento.py::test_fronteras_minutos_normal[0-0.0]
PASSED                                                     [ 76%]
test_estacionamiento.py::test_fronteras_minutos_normal[15-0.0]
FAILED                                                     [ 80%]
test_estacionamiento.py::test_fronteras_minutos_normal[16-20.0]
PASSED                                                     [ 85%]
test_estacionamiento.py::test_fronteras_minutos_normal[60-20.0]
PASSED                                                     [ 90%]
test_estacionamiento.py::test_fronteras_minutos_normal[61-35.0]
FAILED                                                     [ 95%]
test_estacionamiento.py::test_fronteras_minutos_normal[121-50.0]
FAILED                                                    [100%]

=============================================================
FAILURES
=============================================================

_____________________________________________________________
test_71_minutos_35
_____________________________________________________________

sistema = <estacionamiento.Estacionamiento object at
0x7f50936747d0>

   def test_71_minutos_35(sistema):
       #Caso 2
       #R7: $20 por la primera hora y $15 por cada hora adicional
iniciadas
       # Arrange
       minutos = 71
       # Act
       resultado = sistema.calcular_total(minutos, "normal",
False)
```

```
        # Assert
>        assert resultado == pytest.approx(35.0)
E        assert 20.0 == 35.0 ± 3.5e-05
E
E          comparison failed
E          Obtained: 20.0
E          Expected: 35.0 ± 3.5e-05

test_estacionamiento.py:32: AssertionError
_____________________________________________________
test_15_minutos_gratis
_____________________________________________________

sistema = <estacionamiento.Estacionamiento object at
0x7f5093655f30>

    def test_15_minutos_gratis(sistema):
        #Caso 5
        #R5: de 1 a 15 minutos es gratis
        # Arrange
        minutos = 15
        # Act
        resultado = sistema.calcular_total(minutos, "normal",
False)
        # Assert
>        assert resultado == pytest.approx(0.0)
E        assert 20.0 == 0.0 ± 1.0e-12
E
E          comparison failed
E          Obtained: 20.0
E          Expected: 0.0 ± 1.0e-12

test_estacionamiento.py:65: AssertionError
_____________________________________________________
test_61_minutos_costo_35
_____________________________________________________

sistema = <estacionamiento.Estacionamiento object at
0x7f50935f7350>

    def test_61_minutos_costo_35(sistema):
        #Caso 8
        #R7: $20 por la primera hora y $15 por cada hora adicional
iniciadas
        # Arrange
        minutos = 61
        # Act
```

```
        resultado = sistema.calcular_total(minutos, "normal",
False)
        # Assert
>        assert resultado == pytest.approx(35.0)
E        assert 20.0 == 35.0 ± 3.5e-05
E
E          comparison failed
E          Obtained: 20.0
E          Expected: 35.0 ± 3.5e-05

test_estacionamiento.py:95: AssertionError
_____________________________________________________
test_121_minutos_costo_50
_____________________________________________________

sistema = <estacionamiento.Estacionamiento object at
0x7f50933b4e50>

    def test_121_minutos_costo_50(sistema):
        #Caso 9
        #R7: $20 por la primera hora y $15 por cada hora adicional
iniciadas
        # Arrange
        minutos = 121
        # Act
        resultado = sistema.calcular_total(minutos, "normal",
False)
        # Assert
>        assert resultado == pytest.approx(50.0)
E        assert 35.0 == 50.0 ± 5.0e-05
E
E          comparison failed
E          Obtained: 35.0
E          Expected: 50.0 ± 5.0e-05

test_estacionamiento.py:105: AssertionError
_____________________________________________________
test_cliente_frecuente_boleto_perdido
_____________________________________________________

sistema = <estacionamiento.Estacionamiento object at
0x7f5093448c90>

    def test_cliente_frecuente_boleto_perdido(sistema):
        #Caso 13
        #R3 y R8: Boleto perdido sin importar el tipo de cliente
        # Arrange
        minutos = 79
```

```
        # Act
        resultado = sistema.calcular_total(minutos, "frecuente",
True)
        # Assert
>        assert resultado == pytest.approx(300.0)
E        assert 270.0 == 300.0 ± 3.0e-04
E
E          comparison failed
E          Obtained: 270.0
E          Expected: 300.0 ± 3.0e-04

test_estacionamiento.py:150: AssertionError
_____________________________________________________
test_costo_con_decimales
_____________________________________________________

sistema = <estacionamiento.Estacionamiento object at
0x7f509344b5b0>

    def test_costo_con_decimales(sistema):
        #Caso 14
        #R7 y R8: $20 de la primer hora más $15 del inicio de la
segunda menos 10% por cliente frecuente
        # Arrange
        minutos = 97
        # Act
        resultado = sistema.calcular_total(minutos, "frecuente",
False)
        # Assert
>        assert resultado == pytest.approx(31.5)
E        assert 18.0 == 31.5 ± 3.1e-05
E
E          comparison failed
E          Obtained: 18.0
E          Expected: 31.5 ± 3.1e-05

test_estacionamiento.py:163: AssertionError
_____________________________________________________
test_fronteras_minutos_normal[15-0.0]
_____________________________________________________

sistema = <estacionamiento.Estacionamiento object at
0x7f5093665250>, minutos = 15, esperado = 0.0

    @pytest.mark.parametrize("minutos, esperado", [
        (0, 0.0),
        (15, 0.0),
        (16, 20.0),
```

```
        (60, 20.0),
        (61, 35.0),
        (121, 50.0),
    ])

    # Casos frontera parametrizados
    def test_fronteras_minutos_normal(sistema, minutos, esperado):
        resultado = sistema.calcular_total(minutos, "normal",
False)
>       assert resultado == pytest.approx(esperado)
E       assert 20.0 == 0.0 ± 1.0e-12
E
E         comparison failed
E         Obtained: 20.0
E         Expected: 0.0 ± 1.0e-12

test_estacionamiento.py:190: AssertionError
_____________________________________________
test_fronteras_minutos_normal[61-35.0]
_____________________________________________

sistema = <estacionamiento.Estacionamiento object at
0x7f5093487a70>, minutos = 61, esperado = 35.0

    @pytest.mark.parametrize("minutos, esperado", [
        (0, 0.0),
        (15, 0.0),
        (16, 20.0),
        (60, 20.0),
        (61, 35.0),
        (121, 50.0),
    ])

    # Casos frontera parametrizados
    def test_fronteras_minutos_normal(sistema, minutos, esperado):
        resultado = sistema.calcular_total(minutos, "normal",
False)
>       assert resultado == pytest.approx(esperado)
E       assert 20.0 == 35.0 ± 3.5e-05
E
E         comparison failed
E         Obtained: 20.0
E         Expected: 35.0 ± 3.5e-05

test_estacionamiento.py:190: AssertionError
_____________________________________________
test_fronteras_minutos_normal[121-50.0]
_____________________________________________
```

```
sistema = <estacionamiento.Estacionamiento object at
0x7f5093487bb0>, minutos = 121, esperado = 50.0

    @pytest.mark.parametrize("minutos, esperado", [
        (0, 0.0),
        (15, 0.0),
        (16, 20.0),
        (60, 20.0),
        (61, 35.0),
        (121, 50.0),
    ])

    # Casos frontera parametrizados
    def test_fronteras_minutos_normal(sistema, minutos, esperado):
        resultado = sistema.calcular_total(minutos, "normal",
False)
>       assert resultado == pytest.approx(esperado)
E       assert 35.0 == 50.0 ± 5.0e-05
E
E         comparison failed
E         Obtained: 35.0
E         Expected: 50.0 ± 5.0e-05

test_estacionamiento.py:190: AssertionError
=========================================================== short test
summary info ============================================================
FAILED test_estacionamiento.py::test_71_minutos_35 - assert 20.0 ==
35.0 ± 3.5e-05
FAILED test_estacionamiento.py::test_15_minutos_gratis - assert
20.0 == 0.0 ± 1.0e-12
FAILED test_estacionamiento.py::test_61_minutos_costo_35 - assert
20.0 == 35.0 ± 3.5e-05
FAILED test_estacionamiento.py::test_121_minutos_costo_50 - assert
35.0 == 50.0 ± 5.0e-05
FAILED
test_estacionamiento.py::test_cliente_frecuente_boleto_perdido -
assert 270.0 == 300.0 ± 3.0e-04
FAILED test_estacionamiento.py::test_costo_con_decimales - assert
18.0 == 31.5 ± 3.1e-05
FAILED
test_estacionamiento.py::test_fronteras_minutos_normal[15-0.0] -
assert 20.0 == 0.0 ± 1.0e-12
FAILED
test_estacionamiento.py::test_fronteras_minutos_normal[61-35.0] -
assert 20.0 == 35.0 ± 3.5e-05
```

FAILED
test_estacionamiento.py::**test_fronteras_minutos_normal[121-50.0]** -
assert 35.0 == 50.0 ± 5.0e-05

-RESPUESTAS DE ANÁLISIS.

1. ¿Qué diferencia hay entre un caso normal y un caso frontera?
Un caso normal es aquel que está destinado a pasar ya que se encuentra dentro de los parámetros de que se ejecute la instrucción debida correctamente, mientras que en el caso frontera está en el límite de los parámetros para saber si la instrucción se debe de ejecutar.

2. ¿Cuál de sus pruebas considera más importante y por qué?
Boleto perdido con el cliente frecuente ya que mete dos características a considerar, el factor de cliente frecuente ya que necesita descuento pero debe de considerar una excepción de cuando pierde el boleto

3. ¿Encontró algún comportamiento de la implementación que no coincida con el modelo?
Existen 6 pruebas que fallan:

- test_estacionamiento.py::test_71_minutos_35 FAILED
- test_estacionamiento.py::test_61_minutos_costo_35 FAILED
- test_estacionamiento.py::test_121_minutos_costo_50 FAILED
- test_estacionamiento.py::test_costo_con_decimales FAILED
- test_estacionamiento.py::test_fronteras_minutos_normal[61-35.0] FAILED
- test_estacionamiento.py::test_fronteras_minutos_normal[121-50.0] FAILED

Ya que consideran la división entera, por lo tanto no consideran las horas adicionales.

- test_estacionamiento.py::test_15_minutos_gratis FAILED: No considera el caso frontera donde debe ser igual o menor que 15.

- test_estacionamiento.py::test_cliente_frecuente_boleto_perdido FAILED: No se verifica si el boleto está perdido, por lo tanto aplica el descuento aun así.

4. ¿Una suite con 100 % de pruebas aprobadas demuestra que el programa es correcto? Explique.
No, lo único que demuestra es que lo que se ha probado rectifica que el programa pasa esos casos, pero puede haber casos no contemplados que hagan que el programa se rompa o dé una salida ambigua.

5. Si una IA generará automáticamente 50 pruebas, ¿qué tendría que revisar una persona antes de confiar en ellas?
Que vayan acorde a los requisitos del programa, que no solo tome como base el programa, sino casos externos del comportamiento implementado en el programa.